



DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS

College of Arts and Sciences | University of the Philippines Visayas

Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020

Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

Evaluator

CMSC 124 Lab 3

Prepared by: Rene Andre Bedonia Jocsing

Published: October 09, 2026

Deadline: December 09, 2026

Your scanner reads text and your parser builds trees. Trees don't do anything. They're data structures sitting in memory. This activity is where you make them compute by building an **evaluator** that walks a tree and produces a value.

This is the moment Dr. Frankenstein throws the switch. No lightning required, though it does help the atmosphere. It's also the activity where your tests get better. The numbers your evaluator produces are determined by arithmetic. Your expectations can finally be checked against something other than your own earlier output. This activity follows Chapter 7 of Nystrom's *Crafting Interpreters*, "Evaluating Expressions."

Background

Take the expression from last month:

1 2 + 3 * 4

Your parser built a tree that puts the multiplication deeper than the addition:

1 +
2 / \
3 2 *
4 / \
5 3 4

Evaluating it means walking that tree and computing 14. The order falls out of the sequence. Evaluate the leaves, then work up, applying each operator to the values its children produced. Each node evaluates its children before doing its own work. That's a **post-order traversal**. It's how you'd evaluate the expression by hand. It generalizes to every construct you'll add for the rest of the semester.

Representing Values

Before you can compute values, you have to decide how to hold them. That's a stranger problem than it looks, so start with the problem.

Your language is dynamically typed. That was a course requirement in Lab 1. It means a single variable can hold a number now and a string ten lines later, with nothing in the source announcing the

change. So when your evaluator computes something and needs somewhere to put it, that somewhere has to be able to hold a number, a string, a boolean, or your language's nil, and it can't know which until the program runs.

Now look at your host. Most of the languages in the pool are statically typed, which means every variable in your interpreter has to have one type fixed at compile time. You're being asked to store a value whose type isn't known until runtime, in a slot whose type must be known before runtime. That tension is the whole problem. Every host solves it a different way.

Whatever you choose has to give you three things:

1. **Uniform storage.** One host type that any of your language's values can be put into, so your evaluator's functions have something to return.
2. **Runtime interrogation.** A way to ask a value what it currently is, since + needs to know whether it's adding numbers or concatenating strings.
3. **Conversion in both directions.** Getting a host `double` into your value type and back out again, because arithmetic happens on host numbers.

With the question stated, the host answers make sense as answers. C# has `object`, the root every type inherits from. Kotlin has `Any?`, with the `?` admitting nil. Dart has `Object?`. Those three lean on a universal base type and a runtime type check.

Rust and C++ want a tagged union instead, a type that holds exactly one of several alternatives along with a tag saying which: a Rust `enum` with a variant per value type, or `std::variant` in C++. You get interrogation for free, since matching on the tag is how you read it at all, and the compiler will tell you when you've forgotten a case.

Go has `interface{}`, though a small interface with a method or two, or a struct with an explicit tag, is usually clearer than the empty one. Julia is already dynamically typed, so the problem mostly dissolves and you can store values directly.

Pick the idiom your host language encourages, and be ready to defend it. A Rust group that reaches for `Box<dyn Any>` and downcasts everywhere has rebuilt Java's answer inside a language that handed them a better one, and is now fighting its own compiler for no reason.

Evaluator Architecture

Several structures work here. Which one is right depends on what you did in Lab 2.

If you built a visitor in Lab 2, extend it. This is the month that pattern earns its ceremony. Write a second visitor beside your printer, an evaluator with one visit method per node type, each returning a value. Your node classes don't change at all. That was the promise the pattern made you last month. This is it being kept.

If you put behavior on the nodes themselves, give each node an `evaluate` method that returns a value. That's the interpreter pattern. It's shorter, at the cost of mixing tree structure with tree semantics.

If your host language has pattern matching, a match over node types is usually the clearest thing you can write. It's what Rust, Kotlin, and Julia groups should reach for first. Function tables and closures per node type also work.

Whatever the implementation, the principle doesn't move. Every node type gets logic that knows how to evaluate that construct, your evaluation code mirrors your grammar, and errors propagate out cleanly.

Evaluating Each Kind of Expression

Literals are the easy case. The value was determined during scanning, so return it. No computation.

Groupings are nearly as easy. Evaluate the inner expression and return its value. The parentheses did their work back in Lab 2 by building the tree the way they did. They carry no meaning at runtime.

Unary expressions evaluate their operand, then apply the operator. For arithmetic negation, check that the operand is a number first. For logical not, you need to have decided what counts as true in your language.

Binary expressions evaluate the left operand, then the right, then apply the operator. Arithmetic works on numbers. Comparison works on numbers and returns a boolean. Equality works on any pair of types. The interesting one is `+`, which many languages overload to concatenate strings. If yours does, decide what `"a" + 1` means, and “it’s an error” is a perfectly good answer as long as you commit to it.

Order matters here even when it looks like it shouldn’t. Evaluating left before right is a decision. It’s invisible only while operands are pure. The moment an operand can do something, the order becomes observable. Look ahead to Lab 5, where you’ll have functions:

```
1 print f() + g();
```

If `f` prints “first” and `g` prints “second”, then a left-to-right evaluator prints them in that order and a right-to-left one prints them backwards, on identical source. Both produce the same sum. Neither is wrong in any deep sense. Languages disagree. Java and C# fix left to right, while C and C++ left it unspecified for decades and compilers differed. Pick one now, while your own consistency is the only thing depending on it, and write it in your `README.md`. Discovering in November that your evaluator’s order was accidental is a bad month.

Truthiness

Most dynamically typed languages sort every value into true-ish or false-ish so that values other than booleans can be used in conditions and with `!`. The rules are a design choice. A common minimal one is that false and nil are falsey and everything else is truthy, the rule Lox uses. Some languages also treat zero, the empty string, or empty collections as falsey. Python does. C does something else again.

Simplicity has its virtues, but pick on purpose and write the rule in your `README.md`. In Lab 5, every `if` and `while` in your language will depend on it.

Runtime Errors

Some trees can’t be evaluated. Negating a string, adding a number to a boolean, comparing a string with a number. All three are **runtime errors**. The word runtime is the important part. Your scanner and parser can’t catch them. Nothing about `-x` is wrong. Only the value that shows up at runtime is.

When one happens, stop evaluating the current expression, report the problem with its line number, and don’t take the process down with it. That last part matters most for the REPL. Someone who typed a bad expression should be able to fix it and keep working. This is also where the third exit code finally earns its place. A file your interpreter refused before running anything exits 65. A file that started and then hit a runtime error exits **70**.

Learning Objectives

By the end of this activity, you should be able to:

- Evaluate an AST with a post-order traversal, and explain why the order is forced by the tree
 - Choose a runtime value representation that suits your host language, and justify it against the alternatives
 - Implement arithmetic, comparison, equality, and logical negation with runtime type checking
 - Define and defend a truthiness rule for your own language
 - Distinguish static errors from runtime errors, and return the exit code that matches
 - Write inline tests in the *Crafting Interpreters* style, where one file holds a program, its expected output, and its expected exit status
-

Task

Extend your interpreter so it computes values.

The tested path is `./run --eval <path-to-source-file>`, which evaluates each expression in the file and prints its value, one per line. The demonstrated path is the REPL, which now prints values. Your `--tokenize` and `--parse` flags stay exactly as they were, and `tests/lab1/` and `tests/lab2/` have to keep passing.

Required Features

Expression Evaluator

Building on your parser, evaluate:

- Literal values: numbers, strings, booleans, and your language's `nil`
- Grouped expressions
- Unary operators: arithmetic negation and logical not
- Binary arithmetic, including whatever `+` means for strings in your language
- Comparison operators, returning booleans
- Equality and inequality across any pair of types
- Anything else your language's grammar already produces

Value Printing

Your evaluator's output is now what the harness compares, so how a value prints is part of your specification. Three decisions go in your `README.md`:

How do numbers print? If your values are all double-precision floats, `2 + 3` naturally comes out as `5.0`. Trimming a trailing `.0` so it prints `5` is friendlier and is what `jlox` does. Either is fine. Being inconsistent is not.

How does `nil` print? `nil`, `null`, and `nothing` are all reasonable, and it should be one of them every time.

How do strings print? The usual answer is that the value prints without quotes, so `"hi" + " there"` prints `hi there`.

Freeze these before you write tests. Changing them later means editing every annotation in the folder.

Runtime Error Handling

Detect and report:

- Arithmetic or comparison on operands of the wrong type
- Negating something that isn't a number
- Division by zero, if your language treats it as an error

Every runtime error prints its message with a line number on `stderr`, exits 70 in file mode, and returns the prompt in the REPL without killing the session. Something like the textbook's format:

```
1 [line 1] Runtime error: Operands must be numbers.
```

Division by zero is a design decision. IEEE 754, the floating-point standard that your host language's numbers almost certainly implement, says that dividing a positive number by zero produces infinity. So a language that just hands the operands to its host's division operator inherits that answer, and can print `Infinity` and exit 0 with a straight face. A language that rejects it raises a runtime error and exits 70. Both are defensible, both are common in the wild, and I'll ask which one you chose and why. What isn't defensible is a crash with a host-language stack trace.

Which brings up the general rule for this activity. A host-language exception escaping to the top level is a failure. Your interpreter is a program that runs other programs, and other programs are allowed to be wrong. An unhandled `NullPointerException` or a Rust panic means your interpreter is wrong.

The REPL

The REPL now prints computed values. A runtime error prints its message and returns the prompt. As before, the harness doesn't drive this path, so it's graded at defense, where it's the fastest way for me to probe your truthiness rules and your type checking.

The Run Contract

Same contract, third flag:

```
1 ./run --tokenize tests/lab1/keywords.mylang      # Lab 1
2 ./run --parse   tests/lab2/precedence.mylang     # Lab 2
3 ./run --eval    tests/lab3/arithmetic.mylang     # this activity
```

`--eval` means “evaluate each expression in this file and print its value.” Keep it working after Lab 4, when plain `./run <file>` starts executing statements and only explicit print statements produce output. That divergence is why the flag exists. Without it, Lab 4 would invalidate every test you write this month. You'd lose your regression net at the moment your interpreter gets complicated enough to need one.

All three exit codes are now live. 0 means the file ran, 65 means you rejected it before running anything, and 70 means it started and then hit a runtime error.

Writing Tests

This activity switches to **inline mode**. The switch is a promotion. Your Lab 1 and Lab 2 expectations were regression checks against strings your group invented. The values in this activity are determined by arithmetic, so a test can now state what's correct. A reader can see the claim without opening a second file.

It's the same convention *Crafting Interpreters* uses for its own test suite. One file is one test case: the program, the output it should produce, and the exit code it should end on, all together.

The Manifest

Create `tests/lab3/manifest.json`:

```
1 {
2   "ext": ".mylang",
3   "flag": "--eval",
4   "mode": "inline",
5   "comment_prefix": "//"
6 }
```

The new field is `comment_prefix`. It matters because annotations live inside comments, which means the harness has to know what a comment looks like in the language you invented. It defaults to `//`. If your language uses `#`, say so. If it has more than one comment token, list them all, and note that tokens are matched longest first, so a `//` token can't shadow a `///` one:

```

1 {
2   "comment_prefix": ["#", "--"]
3 }

```

If your comments are bracketed and don't run to the end of the line, list the closing token too and it gets stripped off the annotation:

```

1 {
2   "comment_prefix": "(*",
3   "comment_suffix": "*)"
4 }

```

There's no comment syntax that forces you out of inline mode. Setting `comment_prefix` to nothing at all makes the harness stop with a configuration error.

The Three Annotations

`expect`: checks stdout, line by line, in file order:

```

1 3 + 4      // expect: 7
2 2 * 5      // expect: 10
3 "a" + "b"  // expect: ab

```

`expect runtime error`: checks that the message appears on stderr and that the exit code is 70:

```

1 1 / 0      // expect runtime error: Division by zero.

```

`expect error`: does the same for a static error caught before execution starts, and expects exit code 65:

```

1 (1 + 2     // expect error: Expect ')' after expression.

```

Diagnostics are checked against stderr. That's where the run contract puts them. Program output and error messages are different streams and the harness treats them that way.

Three Mechanical Facts to Know Before You Write Fifty of These

The stdout comparison is exact, in order, line by line. The stderr comparison is a **substring** check, so the harness looks for your annotation's text anywhere in stderr. That difference has a practical consequence. Annotate the message only, never the line prefix. Write `// expect runtime error: Operands must be numbers.` and your test survives your reformatting the [line 3] prefix later. Write the prefix into the annotation and it doesn't.

Put one error annotation in a file. Execution stops at the first runtime error, so stderr ends up holding one message and there's nothing for a second annotation to match.

The failure you get from writing two is easy to misread, so here it is in full. Suppose the file says:

```

1 1 / 0      // expect runtime error: Division by zero.
2 "a" - 1    // expect runtime error: Operands must be numbers.

```

The harness collects both messages and joins them with a newline into a single string, then looks for that whole string inside `stderr`. Your interpreter never reached the second line. It reported the division by zero, printed one message, and exited 70. So the harness searches a one-line `stderr` for a two-line block, fails to find it, and reports a mismatch that prints both lines under “expected”. The report makes it look like your first error message came out wrong. It was fine all along. The second annotation is what broke the test.

The exit code has the same problem. A test file carries one expected exit code. Each error annotation overwrites it as the harness reads down the file. `expect error:` sets 65, `expect runtime error:` sets 70, and whichever the harness sees last is the one it asserts. Mix the two in one file and you have written a test that can't pass, since you're now asserting a static error's message against a runtime error's exit code.

Mixing `expect:` lines with an error annotation is fine, as long as you remember that the program stops where it fails. Output produced before the failing line still has to match, line for line:

```
1 1 + 1          // expect: 2
2 10 / 0         // expect runtime error: Division by zero.
3 "never"        // expect: never
```

The third line is a bug in the test. Evaluation dies on line 2, so `never` is never printed, and you get a `stdout` mismatch stacked on top of whatever else is wrong.

All of which means an error test is a one-case file. That's the intended arrangement. Files are cheap, the folder is meant to fill up over the month. A file called `divide_by_zero.mylang` containing one division by zero tells the next reader what broke before they open it.

Annotations can sit at the end of the line they describe or on the line after it. Trailing is usually better, since it keeps the claim next to the code. Line shifting, the thing that pushed Labs 1 and 2 into sidecar mode, doesn't bite here because none of these expectations mention a line number.

A Complete Test File

`tests/lab3/truthiness.mylang:`

```
1 !true          // expect: false
2 !false         // expect: true
3 !nil           // expect: true
4 !"hello"       // expect: false
5 !0             // expect: false
```

That last line is the interesting one. Its expected value depends entirely on the truthiness rule you chose. If zero is falsey in your language, the answer is `true` and this file documents that decision more clearly than a paragraph in your README ever will. Tests are specification you can execute.

Running the Harness Yourself

```
1 curl -sSL https://raw.githubusercontent.com/WhiteLicorice/cmsc-124-harness/v1.1/run_tests.py
   ↪ -o run_tests.py
2 ./build.sh
3 python run_tests.py tests/lab1
4 python run_tests.py tests/lab2
5 python run_tests.py tests/lab3
```

Use `python3` on Linux and macOS, and `Git Bash` on Windows. Each test file still gets 15 seconds.

Inline mode has a trap the sidecar folders didn't. A file with no annotations at all passes trivially. The harness expects no output and exit 0, so a program that prints nothing and exits cleanly satisfies it. A typo in `expect`: doesn't fail loudly. It just stops testing anything. When you add a test, watch the passing count go up by one, and when you doubt a test, break it on purpose and confirm it goes red.

Continuous Integration

Add the new folder, keep the old ones:

```
1 - run: python3 run_tests.py tests/lab0
2 - run: python3 run_tests.py tests/lab1
3 - run: python3 run_tests.py tests/lab2
4 - run: python3 run_tests.py tests/lab3
```

Before you book a defense, all four folders must be green. By now, the workflow is earning its keep. This is the first activity where you'll touch the parser to add evaluation hooks, and `tests/lab2` is what tells you whether the touch was harmless.

Implementation Notes

The Evaluator Itself

The whole evaluator is one function that asks which node type it's holding and does the matching thing. Written as a dispatch, so it reads the same whether your host spells it as a visitor, a match, or a chain of type tests:

```
1 function evaluate(node):
2     if node is Literal : return node.value           // already a value, done
3     if node is Grouping : return evaluate(node.expression)
4
5     if node is Unary:
6         right = evaluate(node.right)                 // operand first
7         if node.operator is "-":
8             checkNumberOperand(node.operator, right)
9             return -right
10        if node.operator is "!":
11            return not isTruthy(right)
12
13    if node is Binary:
14        left = evaluate(node.left)                     // left, then right
15        right = evaluate(node.right)
16        if node.operator is "-":
17            checkNumberOperands(node.operator, left, right)
18            return left - right
19        if node.operator is "<":
20            checkNumberOperands(node.operator, left, right)
21            return left < right
22        if node.operator is "==":
23            return isEqual(left, right)                 // no check, any types
24        if node.operator is "+":
25            if both are numbers : return left + right
26            if both are strings : return left + right
27            throw RuntimeError(node.operator, "Operands must be two numbers or two
↪ strings.")
```

Read the recursion in that. Every branch that has operands evaluates them by calling `evaluate` on itself first, and only then does its own work. That's the post-order traversal. You didn't have to write a traversal to get it. The tree's structure supplies the order.

Where the Type Checks Go

Every operator that needs numbers should check its operands and raise a runtime error naming what it wanted. Write that check once as a helper and call it from each operator. Repeating the same three lines eleven times invites drift.

```
1 function checkNumberOperands(operator, left, right):
2     if left is a number and right is a number : return
3     throw RuntimeError(operator, "Operands must be numbers.")
```

The helper takes the operator's **token**. The symbol alone can't report a line number. Lab 2 asked you to store the token on the node for this reason. This is the first place it pays off. A `RuntimeError` carrying a token can produce `[line 3] Runtime error: Operands must be numbers.` A `RuntimeError` carrying the string `"-"` can produce only the second half. The student debugging a fifty-line file won't thank you.

Equality is the exception. `==` should work on any pair of values and return false for mismatched types, as most dynamically typed languages do. Decide whether `1 == "1"` is false or an error, and write it down.

Unwinding a Runtime Error

You're deep in a recursive traversal when the error happens. Every frame above you is mid-computation. Exceptions are the pragmatic tool. Define one runtime error type carrying a message and a token, throw it from the failing operator, and catch it at the top of the evaluation entry point. There you print it and either exit 70 or return to the REPL prompt. In Rust, thread a `Result` through and let `?` do the unwinding. Either way, the catch happens in exactly one place. Everything between the throw and the catch stays ignorant of errors.

Keeping REPL and File Mode Honest

Both paths should share the evaluator and differ only in what they do with a value and an error. In file mode, a value prints and an error exits 70. In the REPL, a value prints, an error prints, and the loop continues. If you find yourself writing evaluation logic twice, one of the copies is going to drift. It'll be the one you didn't test.

Common Pitfalls

- A host-language exception escaping to the top level. Wrong input to your language should never produce a stack trace from your host.
- Inconsistent number formatting between the REPL and file mode, or between operators. The harness compares strings, so 5 and 5.0 are different.
- Truthiness implemented inline in three places with three slightly different rules. Write one function.
- Equality by host reference, so `"ab" == "ab"` comes out false in a host language where string identity isn't string equality.
- Forgetting to evaluate operands before checking their types, or checking types before evaluating and thereby type-checking a tree. The check must run on a value.
- Exit 65 for a runtime error. A file your parser accepted and then died running exits 70. Getting this backwards fails tests whose stdout is otherwise perfect.
- Losing line numbers. If your error says "Operands must be numbers" with no position, it's a worse error than the one you replaced.
- An annotation typo that quietly stops testing anything. Confirm the passing count changes when you add or break a test.

- Breaking `tests/lab2` while adding evaluation to the parser, and only finding out when CI turns red. That's the system working, but locally is cheaper.
-

Testing Strategy

Your `tests/lab3/` folder should cover at least these, with a file or a labeled group of files for each:

- Each literal type, evaluating to itself
- Arithmetic at several precedence levels, with an answer you computed by hand
- String behavior for `+`, whatever you decided it should be
- Comparison operators, including the boundary cases where `<` and `<=` differ
- Equality across matching types and across mismatched types
- Logical not, including on non-boolean values, which is your truthiness rule under test
- Grouping that changes the result
- A long mixed expression whose value you worked out on paper first

Then the runtime errors, one per file, each with its own `expect runtime error:` annotation:

- Arithmetic on a string
- Comparison between a string and a number
- Negating something that isn't a number
- Division by zero, if that's an error in your language (if it isn't, test the value it produces instead)

Then at least one static error with `expect error:`, to prove your parser's exit 65 path still works now that 70 exists. The two are easy to confuse in the code and the tests are how you find out you did.

Compute expected values yourself before you run anything. The temptation with inline mode is to run the file, read the output, and paste it in as the annotation, which turns a test into the regression check you just graduated from. If you already know $3 + 4 * 2$ is 11, write 11 and then find out whether your interpreter agrees.

Deliverables

Your group's GitHub repository, presented during appointments, with a clean incremental commit history showing who wrote what, must contain:

1. Your evaluator, plus whatever value representation it needs, documented where the code doesn't explain itself
2. A working scanner and parser, with `--tokenize` and `--parse` intact
3. `build.sh` and `run` at the root, executable, supporting all three flags and the no-argument REPL
4. `tests/lab3/` with `manifest.json` and your annotated test files
5. `.github/workflows/test.yml` running all four lab folders, green on the commit you defend
6. A `README.md` whose Semantics sections, per the specification template, record your value representation, your truthiness rule, your value printing format, your division-by-zero decision, and what `+` does with strings

Then, each member submits, individually after the laboratory defense through email: a short `reflection.txt` covering what broke, what you fixed, and what you learned, plus a short `peer.txt` with your honest assessment of how your groupmates, including yourself, worked during the activity. Adhere to the following subject line: [CMSC 124 Lab] Lab 3: LastName, Initials, for example: [CMSC 124 Lab] Lab 3: Sanchez, SM. Include a link to your group's GitHub repository in the email. If even one member of a group fails to submit their individual deliverables, no final grade for the activity may be released for all members of the group.

Academic Honesty

Using large language models to generate wholesale vibe-coded submissions is cheating, subject to failure in the course and harsh disciplinary action. This activity is where generated code starts to show. The design decisions (truthiness, numeric formatting, what + means, what division by zero does) have to agree with each other across the evaluator, the tests, and the README. Code you didn't write tends to disagree with itself. The defense is ten minutes of that question.

Important Dates

Progress reports and the laboratory defense may be booked only during the dates and hours defined in the syllabus. Book ahead through the booking page on the course site.

Activity	Monday	Tuesday
Week 1 Progress Report	Oct 12	Oct 13
Week 2 Progress Report	Oct 19	Oct 20
Week 3 Progress Report	Oct 26	Oct 27
Week 4 Laboratory Defense	Nov 2	Nov 3

Appointments need my verification before they count. If a date falls on a holiday or a suspension of classes, the syllabus allows a recorded progress report or defense for that seven-day period.

For progress reports, Week 1 wants your value representation chosen and defended, with literals and arithmetic evaluating. Week 2 wants every operator working with type checks. The inline test folder grows alongside the code. Week 3 wants runtime errors landing on the right exit codes, green CI across all four folders, and your documented design decisions.

Grading Rubric

Criteria	Excellent (90-100%)	Good (75-89%)	Fair (60-74%)	Poor (0-59%)
Implementation Correctness (25%)	Every required feature of the activity works, including edge cases; the run contract is honored exactly, with program output on stdout, diagnostics on stderr, and the right exit code for each class of failure.	Core features correct with minor gaps in fringe cases; streams and exit codes right.	Core features work, but several requirements are missing or wrong; stream separation or exit codes are inconsistent.	Fails the activity's core requirement, ignores the run contract, or crashes with host-language errors on malformed input.
Testing and CI (20%)	Committed tests cover every required feature and its failure cases; expectations were reasoned out; every lab folder is green in CI on the defended commit.	Good coverage of the main features with at least one failure case; CI green.	Only happy paths tested, or an earlier lab folder left failing.	Almost no tests, no manifest, or no working workflow.
Code Engineering (10%)	Clean separation between pipeline stages; shared logic written once; constructs idiomatic for the host language; comments explain why.	Mostly clean structure with minor duplication or naming problems.	Monolithic functions; repeated logic; one stage leaking into another.	Unreadable, or the code contradicts the documented language design.
Collaboration (20%)	Professional Git usage: atomic, semantic commits from every member spread across the accomplishment period; tests and features arrive together; evidence of review or pair programming.	Consistent use of version control; adequate commit messages; evidence of a structured team workflow.	Inconsistent Git usage; large code dumps instead of incremental progress; vague commit messages.	Minimal use of version control; repository lacks history, or history was rewritten or crammed.
Technical Defense (25%)	Every member traces their own code over fresh input, handles what-if questions, and defends the language design decisions behind the implementation.	Clear explanation of the implementation; both members participate meaningfully; logic delivery is sound.	Unclear explanations; uneven participation; struggles with code-tracing questions.	Unprepared; cannot explain the pipeline or defend design decisions.